

Instruction of New Fitting Code

Yao Fu

(Dated: March 2025)

I. Introduction

In this note, we illustrate how to compile and use the new fitting code written in C++.

II. Compilation in MSU HPCC

Most of the environment in MSU HPCC cluster has been setup. One can use the following script to setup the environment:

```
source /mnt/home/fuyao3/setup.sh
```

Only the eigen and python part need to be setup by users locally. For the installation of eigen package, see section III E. For the python part, see section III I. Note that the code can be compiled only if the python numpy is installed locally.

III. Compilation in other cluster

The code depends on ROOT and CMAKE. We recommend to use the GCC version $\geq 8.3.0$, ROOT version $\geq 6.20.06$ and CMAKE version ≥ 3.15 . Some lower version of ROOT could also work for this code. Except the ROOT and CMAKE, the code also depends on the ApplGrid, FastNLO, Hoppet, LHAPDF, Eigen, yaml-cpp, BAT, APFEL, APFELxx, APFELgrid, and Python. This code uses many functions of ApplGrid 1.5.xx version. However, the code structure of ApplGrid 1.6.xx has big changes, so the ApplGrid 1.6.xx doesn't work for this code. We recommend using the FastNLO version $\geq 2.3.1$, and LHAPDF version $\geq 6.2.3$, Hoppet version $\geq 1.2.0$, and Eigen version $\geq 3.4.0$.

A. Installation of LHAPDF

Download page: <https://lhapdf.hepforge.org/downloads/>

Installation:

```
./configure --prefix=/path to install
make -j16
make install
```

B. Installation of Hoppet

Download page: <https://hoppet.hepforge.org/downloads/hoppet-1.2.0.tgz>

Installation:

```
./configure --prefix=/path to install
make -j16
make install
```

C. Installation of ApplGrid

Download page: <https://applgrid.hepforge.org/downloads/applgrid-1.5.46.tgz>

Before installing ApplGrid, you should first install LHAPDF and HOPPET, and set the necessary PATH and LD_LIBRARY_PATH to environment.

```
#LHAPDF
export LHAPDF_ROOT_DIR=/path to LHAPDF/
export LHAPDF_PDF_DIR=${LHAPDF_ROOT_DIR}/share/LHAPDF
export PATH=$PATH:${LHAPDF_ROOT_DIR}/bin
export LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:${LHAPDF_ROOT_DIR}/lib
#HOPPET
export Hoppet_ROOT_DIR=/path to Hoppet/
export PATH=$PATH:/path to Hoppet/bin
export LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:/path to Hoppet/lib
```

Then copy the patch to your ApplGrid source code directory:

```
cp -r FittingCode/src/MCFMPatch/ApplGrid/* /path to ApplGrid source code/
```

Installation:

```
./configure --prefix=/path to install
make -j16
make install
mkdir /install area of ApplGrid/share
mkdir /install area of ApplGrid/share/applgrid
cp /ApplGrid source code/share/* /install area of ApplGrid/share/applgrid
```

Sometimes the installation will fail. You can try to install automake and libtool by yourself before running the configure of ApplGrid.

For automake,

Download page: <https://ftp.gnu.org/gnu/automake/automake-1.15.tar.gz>

Installation and set environment:

```
./configure --prefix=/path to install
make -j16
make install
export PATH=/path to automake/bin:$PATH
```

For libtool:

Download page: <https://mirror.cyberbits.eu/gnu/libtool/libtool-2.4.tar.gz>

Installation and set environment:

```
./configure --prefix=/path to install
make -j16
make install
export PATH=/path to libtool/bin:$PATH
export LD_LIBRARY_PATH=/path to libtool/lib:${LD_LIBRARY_PATH}
export LIBTOOL=/path to libtool/bin/libtool
```

Before rerun the configure, run the following line first:

```
autoreconf --force --install
```

After the above steps, compile the ApplGrid again.

D. Installation of FastNLO

Download page: https://fastnlo.hepforge.org/code/v23/fastnlo_toolkit-2.3.1pre-2411.tar.gz

Then copy the patch to your FastNLO source code directory:

```
cp -r FittingCode/src/MCFMPatch/FastNLO/* /path to FastNLO source code/
```

Installation:

```
./configure --prefix=/path to install  
make -j16  
make install
```

E. Installation of Eigen

Download page: <https://gitlab.com/libeigen/eigen>

Installation:

```
mkdir build  
cd build  
cmake .. -DCMAKE_INSTALL_PREFIX=.  
make -j16  
make install
```

F. Installation of yaml

Download page: <https://github.com/jbeder/yaml-cpp>

Installation:

```
mkdir build
cd build
cmake .. -DCMAKE_INSTALL_PREFIX=. -DBUILD_SHARED_LIBS=ON
make -j16
make install
```

For some machine, after installing the code, there will be a `lib` directory there, but for some other machine, it will be `lib64`. You need to make sure the correct path be added into the dynamic link library.

G. Installation of BAT (optional)

Download page: <https://github.com/bat/bat>

Installation:

```
./autogen.sh
./configure --prefix=/path to install
make -j16
make install
```

H. Installation of APFEL, APFELgrid, APFELxx

Download page:

apfel: <https://github.com/vbertone/apfelxx>

apfelxx: <https://github.com/scarrazza/apfel>

APFELgrid: <https://github.com/nhartland/APFELgrid>

Then copy the patch to your APFEL source code directory:

```
cp -r FittingCode/src/MCFMPatch/apfel/* /path to apfel source code/
```

1 Installation of APFEL

Install the APFEL first. If you don't want to include Python, please set

```
-DAPFEL_ENABLE_PYTHON=OFF
```

when doing cmake. If you want to include Python, before compiling the APFEL, install the SWIG first. For installing the SWIG, see section [III I](#)

```
cmake .. -DCMAKE_INSTALL_PREFIX=.
-DAPFEL_Python_SITEARCH=/absolute path of your build directory
-DDATA_PATH=${PWD}/../src/HELL/data/

make -j16
chmod -r lib/libAPFEL.so.0.0.0
chmod -r lib/libAPFELevol.so.0.0.0
make install
chmod +r lib/libAPFEL.so.0.0.0
chmod +r lib/libAPFELevol.so.0.0.0
```

2 Installation of HELL, HELLx (optional)

To perform the small- x resummation, the HELL grid is needed.

Installation of HELL grid:

<https://www.roma1.infn.it/~bonvini/hell/>

<https://www.roma1.infn.it/~bonvini/hell/downloads/HELLx-data.v3.tgz>

```
mkdir /path to apfel/src/HELL/data/apfel
cp -r /path to HELL/data/* /path to apfel/src/HELL/data/apfel
```

To create new HELL grid, we should download the code for advanced users only:

<https://www.roma1.infn.it/~bonvini/hell/downloads/HELL.v3.1.tgz>

<https://www.roma1.infn.it/~bonvini/hell/downloads/HELL-data.v3.tgz>

<https://www.roma1.infn.it/~bonvini/hell/downloads/HELL-data-aux.v3.1.tgz>

The advanced HELL code depends on gsl and Cuba. Here is the download page of Cuba:

<https://feynarts.de/cuba/Cuba-4.2.2.tar.gz>

Installation of Cuba:

```
./configure --prefix=/path to install
make #Note that there should not be -j to install
make install
```

After installing the Cuba, setup the environment before compiling the HELL code:

```
#Note that the CUBA_PATH is needed for HELL code, do not use other name
export CUBA_PATH=/path to cuba
export PATH=$PATH:${CUBA_PATH}/bin
export LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:${CUBA_PATH}/lib
```

The default α_S grid points provided by HELL are not enough for our CTEQ-TEA PDF fitting, we need to generate more α_S points.

1. Add more grid points to the following files:

```
./data/LL_nf3.info
./data/LL_nf4.info
./data/NLL_nf3.info
./data/NLL_nf4.info
```

2. Generating new deltaC, deltaP, deltaU files:

```
./examples/create_deltaC_tables -a 0.36 -n 3
./examples/create_deltaC_tables -a 0.36 -n 4
./examples/create_deltaCmass_tables -a 0.36 -n 3
```

```
./examples/create_deltaCmass_tables -a 0.36 -n 4
./examples/create_deltaP_tables -a 0.36 -n 3
./examples/create_deltaP_tables -a 0.36 -n 4
./examples/create_deltaU_tables -a 0.36 -n 3
./examples/create_deltaU_tables -a 0.36 -n 4
to be added more $\alpha_S$ values...
```

After generating new tables, copy the new files to the APFEL directory.

3 Installation of APFELgrid

We can install the APFELgrid after installing the APFEL and ApplGrid. The APFELgrid needs the header files of ApplGrid, so in the very beginning we need to copy all headers of ApplGrid to the ApplGrid install area.

```
cp /ApplGrid source code/src/*.h /ApplGrid install path/
```

By default the APFELgrid should be installed to the same path of APFEL install area, so for installing the APFELgrid, set the APFEL environment at first, then compile by the following command:

```
export PATH=/path to apfel/bin:$PATH
export APPLGRID_ROOT_DIR=/path to ApplGrid/
export LD_LIBRARY_PATH=${APPLGRID_ROOT_DIR}/lib:${LD_LIBRARY_PATH}
export PATH=${APPLGRID_ROOT_DIR}/bin:${PATH}
export PDFPATH='applgrid-config --share'
autoreconf -i
./configure --prefix=$(apfel-config --prefix)
make -j16
make install
```

4 Installation of APFELxx

APFELxx is standalone. For installation:

```
mkdir build
cd build
cmake .. -DCMAKE_INSTALL_PREFIX=.
make -j16
make install
```

I. Installation of SWIG, Python, TensorFlow

Download page of SWIG: <https://github.com/swig/swig>

Installation:

```
./configure --prefix=/path to install
make -j16
make install
```

We recommend using Python version ≥ 3.11 . Some package needs to be installed by ourselves.

```
python3 -m pip install --upgrade pip
python3 -m pip install --user numpy
python3 -m pip install --user scipy
python3 -m pip install --user tensorflow
python3 -m pip install --user tensorflow-probability[tf]
```

J. Installation of PineAPPL

Download page of PineAPPL: <https://github.com/NNPDF/pineappl>

Installation: See the instruction from PineAPPL webpage:

<https://github.com/NNPDF/pineappl/blob/master/docs/installation.md>

If your Rust version is too old, use the following method to update the Rust:

```
curl https://sh.rustup.rs -sSf | sh -s -- -y --profile minimal
source $HOME/.cargo/env
rustup update stable
rustup default stable
export PATH=$HOME/.cargo/bin:$PATH
```

Once update the rust, when login to cluster again, we just need to setup the following:

```
export PATH=$HOME/.cargo/bin:$PATH
```

K. Setup environment

Before the compilation, some of the environment variables need to be setup:

```
#LHAPDF
export LHAPDF_ROOT_DIR=/path to LHAPDF/
export LHAPDF_PDF_DIR=${LHAPDF_ROOT_DIR}/share/LHAPDF
export PATH=$PATH:${LHAPDF_ROOT_DIR}/bin
export LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:${LHAPDF_ROOT_DIR}/lib
#HOPPET
export Hoppet_ROOT_DIR=/path to Hoppet/
export PATH=$PATH:${Hoppet_ROOT_DIR}/bin
export LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:/path to Hoppet/lib
#ApplGrid
export APPLGRID_ROOT_DIR=/path to ApplGrid/
export LD_LIBRARY_PATH=${APPLGRID_ROOT_DIR}/lib:${LD_LIBRARY_PATH}
export PATH=${APPLGRID_ROOT_DIR}/bin:${PATH}
export PDFPATH='applgrid-config --share'
#Fast NLO
export FASTNLO_ROOT_DIR=/path to FastNLO/
export LD_LIBRARY_PATH=${FASTNLO_ROOT_DIR}/lib:${LD_LIBRARY_PATH}
export PATH=/path to FastNLO/bin:${PATH}
#yaml
export YAML_ROOT_DIR=/path to yaml
```

```

export LD_LIBRARY_PATH=/path to yaml/lib64:${LD_LIBRARY_PATH}
#BAT
export BAT_ROOT_DIR=/path to BAT/
export PATH=$PATH:${BAT_ROOT_DIR}/bin
export LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:${BAT_ROOT_DIR}/lib

#apfel APFELgrid swig
export apfelxx_ROOT_DIR=/path to apfelxx/
export PATH=${apfelxx_ROOT_DIR}/bin:$PATH
export LD_LIBRARY_PATH=${apfelxx_ROOT_DIR}/lib:${LD_LIBRARY_PATH}
export PATH=/path to apfel/bin:$PATH
export LD_LIBRARY_PATH=/path to apfel/lib:${LD_LIBRARY_PATH}
export PATH=/path to SWIG/bin:$PATH

#PineAPPL
export PATH=$HOME/.cargo/bin:$PATH
export PineAPPL_ROOT_DIR=/path to PineAPPL/
export LD_LIBRARY_PATH=${PineAPPL_ROOT_DIR}/lib:$LD_LIBRARY_PATH
export PKG_CONFIG_PATH=${PineAPPL_ROOT_DIR}/lib/pkgconfig:$PKG_CONFIG_PATH

```

Note that the XXX_ROOT_DIR is needed. We don't need to setup any environment for Eigen package. The code will automatically find the Eigen when doing the compile. For compile, please usage:

```

cd FittingCode
mkdir build
cd build
cmake ..
make -j install

```

After compile the code, we need to setup the environment variables for running:

```

export PATH=${PATH}:/path to FittingCode/build/bin
export LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:/path to FittingCode/build/lib

```

IV. Input card

In `src/Calculation/Input`, there are examples of all the input cards.

A. Data list

- **GlobalPath:**

- **data:** The global path of data files
- **applgrid:** The global path of the ApplGrid files.
- **fastnlo:** The global path of the FastNLO grid files.
- **pineappl:** The global path of the PineAPPL grid files.

- **DataInformation:**

- **A Node Name:**

The node name will be used for giving a specific name to a data set. In the code, the node name will be the input parameter of the constructor of the Dataset class. For example, if the node name is HERAIpII, it is equivalence to have

```
AllData.at(0) = new Dataset("HERAIpII");
```

After running the code, there will be an output file named `HERAIpII.dta`.

- * **Path:** The file name of the data set.

- * **DataType:** DIS, DrellYan, or Jet.

- * **CalcMode:**

- CTEQ: Internal package for doing DIS calculations.
- CTEQ DY: Internal package for doing DrellYan calculations.
- ApplGrid: Convolute with ApplGrid.
- FastNLO: Convolute with FastNLO.

- * **Collider:**

LHC, or Tev (which means Tevatron). This option only affects the ApplGrid and FastNLO calculation.

- * **subType:**

Currently for all the data that CT18 included, the subType is the same as the name of the data, as the data file format is different from each other. In the future, we will have unified format of data. As a result, the subType will be CommonDIS, CommonDY, or CommonJet.

- * **ApplGrid:**

The ApplGrid names of the data set. All the names need to start on a

new line and include indentation (two spaces). For example,

```

    ApplGrid:
    - 245_1.root
    - 245_2.root
    - 245_3.root

```

* **ApplRange:**

Specify the range of ApplGrid files used for doing convolution. The input format is the same as **ApplGrid**. For example,

```

    ApplRange:
    - [1, 1, 17]

```

where the first parameter means which ApplGrid, the second and third parameters mean the range of this ApplGrid. The binning index starts from 1.

* **FastNLO:**

The FastNLO grid names of the data set. All the names need to start on a new line and include indentation (two spaces).

* **PineAPPL:**

The PineAPPL grid names of the data set. All the names need to start on a new line and include indentation (two spaces).

* **KFactor:**

The k-factor columns which will be multiplied to the theory prediction. For example,

```

    KFactor: [5, 8]

```

where this means the k-factor of 5th and 8th columns will be used, and they will be multiplied together. The column index starts from 1.

* **Systematic:**

How to calculate the systematic uncertainty shift.

0 means the systematic uncertainty shift is based on data.

$$Shift = \beta * Data \tag{1}$$

1 means the systematic uncertainty shift is based on theory.

$$Shift = \beta * Theory \tag{2}$$

If this flag is empty, the default one is using the theory. For example,

Systematic:

- [0, 0, 0]
- [1, 73, 1]

where the first and second parameter mean the column range of systematic uncertainty, the index starts from 0. The third parameter means which method is used. The new line and indentation are also needed.

* **Normalization:**

Giving the normalization parameter. For example,

Normalization: [[71, N71], [0.9676, 0.1, 1.0]]

where the first two parameters are the parameter ID and parameter name, which should also be in the Parameter.yml file. The last three parameters are the initial value and the penalty parameter **FacNor** and **FacNor4**, respectively. Here the initial normalization value will be override by the one implemented in the Parameter.yml file. The penalty parameter decides the additional normalization penalty which will be added to the final χ^2 .

$$Penalty = FacNor * \left(\frac{Norm - 1}{NormErr} \right)^2 + FacNor4 * \left(\frac{Norm - 1}{NormErr} \right)^4 \quad (3)$$

B. Config

- **QCDParameter:**

- **mc:** The charm quark mass.
- **mb:** The bottom quark mass.
- **mt:** The top quark mass.
- **Qini:** The PDF initial scale.
- **AlphaS:** The strong coupling constant.
- **AlphaS_Q0:** The energy scale of the strong coupling constant.
- **Evolution:** The PDF Evolution package. Hoppet, APFEL, or APFELxx
- **EvolutionOption:** Additional evolution options.

- **Algorithm:**

- **Name:** The minimization algorithm name. Simplex, Migrad, Minimize, Fumili,
- **MaxCall:** Maximum call when minimizing the χ^2 .

- **Tolerance**: The tolerance of the algorithm.
- **LFit**: Maximum call of the minimization algorithm. For each time of call, the NNLO/NLO k-factor will be calculated again.
- **Thread**:
 - **nDISThread**: The number of thread used for calculating the DIS process.
 - **nVBPTHread**: The number of thread used for calculating the DrellYan process.
- **ErrorSet**:
 - **Tolerance**: T^2 of PDF error set.
 - **Tier2Type**: Tier2 penalty type.
 - **CalcLevel**: **Full** means doing χ^2 calculation. **PDFOnly** means not doing χ^2 calculation, but only producing the PDF files.

C. Parameters

- **Parameter**: The initial value of parameters are read from **CentralValue**. Here only the initial value of normalization parameter are read. Example:
 - parameter: [1, A1glu, 0.53101, 0.05, 0.0, 1.0]
 - parameter: [2, A2glu, 3.14810, 0.05, 0.0, 5.0]
 - parameter: [3, A3glu, 3.03140, 0.05, 0.0, 5.0]
 - parameter: [4, A4glu, -1.70494, 0.05, -2.0, 0.0]
 - parameter: [10, gluM, 0.0, 0.01, -0.5, 0.5]
 - parameter: [11, A1uvl, 0.76317, 0.05, 0.0, 1.0]
 - parameter: [12, A2uvl, 3.03609, 0.05, 0.0, 5.0]
 - parameter: [13, A3uvl, 1.50192, 0.05, 0.0, 5.0]
 - parameter: [14, A4uvl, -0.14666, 0.05, -1.0, 0.0]
 - parameter: [15, A5uvl, 1.67108, 0.05, 0.0, 5.0]
 - parameter: [23, A3dvl, 2.61407, 0.05, 0.0, 5.0]
 - parameter: [24, A4dvl, 1.82746, 0.05, 0.0, 5.0]
 - parameter: [25, A5dvl, 2.72032, 0.05, 0.0, 5.0]
 - parameter: [41, A1dou, 0.61791, 0.05, 0.0, 1.0]
 - parameter: [42, A2dou, 0.19493, 0.05, 0.0, 1.0]
 - parameter: [43, A3dou, 0.87087, 0.05, 0.0, 1.0]
 - parameter: [44, A4dou, 0.26669, 0.05, 0.0, 1.0]
 - parameter: [45, A5dou, 0.73317, 0.05, 0.0, 1.0]
 - parameter: [51, A1dpu, -0.02194, 0.05, -1.0, 0.0]
 - parameter: [52, A2dpu, 7.73657, 0.05, 5.0, 10.0]
 - parameter: [54, A4dpu, 0.29223, 0.05, 0.0, 1.0]
 - parameter: [55, A5dpu, 0.64695, 0.05, 0.0, 1.0]
 - parameter: [56, A6dpu, 0.47492, 0.05, 0.0, 1.0]

- parameter: [57, A7dpu, 0.74137, 0.05, 0.0, 1.0]
- parameter: [60, strM, 0.0, 0.05, -0.5, 0.5]
- parameter: [62, A2str, 10.30986, 0.05, 7.0, 12.0]
- parameter: [64, A4str, 0.46599, 0.05, 0.0, 1.0]
- parameter: [65, A5str, 0.22525, 0.05, 0.0, 1.0]
- parameter: [71, N71, 0.9676, 0.01, 0.9, 1.1]
- parameter: [72, N72, 1.0137, 0.01, 0.9, 1.1]
- parameter: [73, N73, 1.0290, 0.01, 0.9, 1.1]

- **PDFParameterization:** Example:

IFun: 1

- **ShapeMatch:** Example:

- parameter: [dv, 981, [11, 12, 23, 24, 25, 0, 0, 0, 0, 0, 0, 0]]
- parameter: [uv, 981, [11, 12, 13, 14, 15, 0, 0, 0, 0, 0, 0, 0]]
- parameter: [gluon, 848, [1, 2, 3, 4, 0, 0, 0, 0, 0, 0, 0, 0]]
- parameter: [dmu, 219, [51, 52, 0, 54, 55, 56, 57, 41, 42, 43, 44, 45]]
- parameter: [dpu, 218, [51, 52, 0, 54, 55, 56, 57, 41, 42, 43, 44, 45]]
- parameter: [str, 220, [51, 62, 0, 64, 64, 65, 65, 0, 0, 0, 0, 0]]

- **CentralValue:** The initial value of parameters are read from here. Example:

- parameter: [dv, 0.1344670, 0.0000000, [0.76317, 3.03609, 2.61407, 1.82746, 2.72032, 0.00000, 0.00000, 0.00000, 0.00000, 0.00000, 0.00000, 0.00000]]
- parameter: [uv, 0.3250827, 0.0000000, [0.76317, 3.03609, 1.50192, -0.14666, 1.67108, 0.00000, 0.00000, 0.00000, 0.00000, 0.00000, 0.00000, 0.00000]]
- parameter: [gluon, 0.3846129, 0.0000000, [0.53101, 3.14810, 3.03140, -1.70494, 0.00000, 0.00000, 0.00000, 0.00000, 0.00000, 0.00000, 0.00000, 0.00000]]
- parameter: [dmu, 0.0000000, 0.0000000, [-0.02194, 7.73657, 4.00000, 0.29223, 0.64695, 0.47492, 0.74137, 0.61791, 0.19493, 0.87087, 0.26669, 0.73317]]
- parameter: [dpu, 0.1291034, 0.0000000, [-0.02194, 7.73657, 4.00000, 0.29223, 0.64695, 0.47492, 0.74137, 0.61791, 0.19493, 0.87087, 0.26669, 0.73317]]
- parameter: [str, 0.0267340, 0.5146522, [-0.02194, 10.30986, 4.00000, 0.46599, 0.46599, 0.22525, 0.22525, 1.00000, 0.00000, 0.00000, 0.00000, 0.00000]]

V. Code structure

A. PDF Parameterization and Evolution

1 PDF Parameterization

PDF Parameterization is based on the gfun structure. The parameterization function is defined in `src/CTEQHelper/gfuns.C` and `include/CTEQHelper/gfuns.h`. The PDF in Q_0 scale is generated by `src/CTEQHelper/PDFEvolve.C` and `include/CTEQHelper/PDFEvolve.h`.

To generate a PDF in Q_0 scale, usage:

```
pdf = new PDFEvolve();
vector<TString> FlavorList = {"dv", "uv", "gluon", "dmu", "dpu", "str"};
vector<int> FlagList = {981, 981, 848, 219, 218, 220};
pdf->SetParameter("dv",    vector<double>{ 0.76317,  3.03609, 2.61407,  1.82746,
2.72032,  0.00000,  0.00000,  0.00000,  0.00000,  0.00000,  0.00000,  0.00000});
pdf->SetParameter("uv",    vector<double>{ 0.76317,  3.03609, 1.50192, -0.14666,
1.67108,  0.00000,  0.00000,  0.00000,  0.00000,  0.00000,  0.00000,  0.00000});
pdf->SetParameter("gluon", vector<double>{ 0.53101,  3.14810, 3.03140, -1.70494,
0.00000,  0.00000,  0.00000,  0.00000,  0.00000,  0.00000,  0.00000,  0.00000});
pdf->SetParameter("dmu",    vector<double>{-0.02194,  7.73657, 4.00000,  0.29223,
0.64695,  0.47492,  0.74137,  0.61791,  0.19493,  0.87087,  0.26669,  0.73317});
pdf->SetParameter("dpu",    vector<double>{-0.02194,  7.73657, 4.00000,  0.29223,
0.64695,  0.47492,  0.74137,  0.61791,  0.19493,  0.87087,  0.26669,  0.73317});
pdf->SetParameter("str",    vector<double>{-0.02194, 10.30986, 4.00000,  0.46599,
0.46599,  0.22525,  0.22525,  1.00000,  0.00000,  0.00000,  0.00000,  0.00000});
pdf->SetInitialgluonMomentum(0.3846129);
pdf->SetInitialStrangeB0(0.5146522);
pdf->Initialize(1, FlavorList, FlagList);
```

The FlavorList and FlagList are used for telling the code which gfun should be used for each flavor. Function SetParameter(TString, vector<double>) is used for setting the parameters for each flavor. The normalization of each flavor will be done after calling the Initialize() function. The function Normalization() is defined for how to do the normalization. Here is an example for doing the normalization:

```

double uvInt = NumberIntegral("uv", ifun);
double dvInt = NumberIntegral("dv", ifun);
uvNorm = 2.0 / uvInt; //!
dvNorm = 1.0 / dvInt; //!
double uvxInt = MomentumIntegral("uv", ifun);
double dvxInt = MomentumIntegral("dv", ifun);
double uvMomentum = uvNorm * uvxInt;
double dvMomentum = dvNorm * dvxInt;
RemM = RemM - uvMomentum - dvMomentum;

double gluonxInt = MomentumIntegral("gluon", ifun);
gluonNorm = gluonMomentum / gluonxInt; //!
RemM = RemM - gluonMomentum;

double StrangeMomentum = RemM / 3.0 * StrangeB0;
double sxInt = MomentumIntegral("str", ifun);
sNorm = StrangeMomentum / sxInt; //!
RemM = RemM - StrangeMomentum;

double LastSeaMomentum = RemM; //ubar+dbar
double LastSeaxInt = MomentumIntegral("dpu", ifun);
dpuNorm = LastSeaMomentum / LastSeaxInt;
dmuNorm = 1.0;

```

2 PDF Evolution

The PDF evolution is based on the Hoppet. An interface code for calling the Hoppet is defined, see `src/CTEQHelper/HoppetInterface.C` and `include/CTEQHelper/HoppetInterface.h`.

To do the PDF evolution, usage:

```

hoppet = new HoppetInterface();
hoppet->SetAlphaSQ0(0.118, 91.19);
hoppet->Initialize(pdf_Q0, 1.29);
hoppet->SetQArray();

```

```

hoppet->SetXArray();
pdsReader* pdftable = global_hoppet->PrepareCTEQTable();

```

The Initialize() function needs a pointer of PDFEvolve class, and the Q_0 scale input. The function PrepareCTEQTable() will handle the full evolution, and output the UPD. Another class pdsReader is defined for doing the interpolation of UPD and getting the PDF values for a given x and Q for each flavor, which is written based on the Fortran CTEQ-TEA fitting code.

3 PDF interpolation

The interpolation class pdsReader is a standalone code for getting the PDF values. An xfxQ(flavor, x, Q) function is defined to match with the function which has the same name in LHAPDF. In this function, the flavor definition is the same as what LHAPDF uses. For example u quark is 2, and d quark is 1.....

The initialization can be done by reading the existing .pds file, or just giving the UPD and AlphaS input.

```

virtual void Initialize(const char* name);
virtual void Initialize(int NfmX, int MxVal, int NX, int NT,
std::vector<double> XV, std::vector<double> qv, std::vector<double> TV,
std::vector<double> AlsCTEQ, std::vector<double> UPD);

```

We can also initialize a pdsReader by clone from the existing pointer. Here are some examples for initializing the pdsReader class:

```

//Initialize from .pds file
m_CTEQPDF = new pdsReader();
m_CTEQPDF->Initialize(PDFFile);

//Initialize by clone
m_CTEQPDF = new pdsReader();
m_CTEQPDF->CopyHeader(pdf);
m_CTEQPDF->InputAlphaS(pdf->AlsCTEQ);
m_CTEQPDF->InputUPD(pdf->UPD);

```

B. Theory Calculation

1 QCD and Electroweak Setting

See the following codes:

```
src/Calculation/QCDSet.C,  
include/Calculation/QCDSet.h,  
src/Calculation/EwkSet.C,  
include/Calculation/EwkSet.h
```

The DIS and VBP processes codes are inheriting the QCDSet and EwkSet:

```
class CalcDIS : public mDisPac, public CTUtil  
class mDisPac : public EwkSet, public QCDSet  
class CalcVBP : public CTUtil, public EwkSet, public QCDSet
```

2 Numerical Utilities

The needed numerical codes are defined in CTUtils class. See src/Calculation/CTUtils.C and include/Calculation/CTUtils.h. The DIS and VBP processes codes are also inheriting the CTUtils.

3 DIS process

A class CalcDIS is defined for doing the calculation of DIS process. This class is rewritten from the Fortran CTEQ-TEA fitting code. All the logic are the same as the Fortran code uses. By reading the information of data table, it can output the consistent results as the Fortran code.

To get the prediction, usage:

```
m_calculation->CalcDIS::InitialCTEQPDF(setting.m_cteqpdf);  
m_calculation->CalcDIS::InitializeData(setting, m_data);  
vector<double> xsec;  
m_calculation->CalcDIS::GetPrediction(xsec);
```

4 VBP process

A class CalcVBP is defined for doing the calculation of Drell-Yan process. This class is rewritten from the Fortran CTEQ-TEA fitting code. All the logic are the same as the Fortran code uses. The calling method is the same as CalcDIS class.

5 ApplGrid

A class ApplGridHelper is defined for reading the ApplGrid files.

To get the prediction from ApplGrid, usage:

```
m_calculation->ApplGridHelper::InitialCTEQPDF(setting.m_cteqpdf);
m_calculation->ApplGridHelper::InitialGrid(GridName);
vector<double> xsec;
m_calculation->ApplGridHelper::DefaultConvoluteMacro(GridName, xsec);
```

6 FastNLO

A class FastNLOHelper is defined for reading the FastNLO tables.

To get the prediction from FastNLO, usage:

```
m_fnlo = new FastNLOHelper(FNLOTableName);
m_fnlo->FastNLOHelper::InitialCTEQPDF(setting.m_cteqpdf);
m_fnlo->SetContributionON(fastNLO::kFixedOrder,1,true);
m_fnlo->SetContributionON(fastNLO::kFixedOrder,2,true);
m_fnlo->SetScaleFactorsMuRMuF(1.0, 1.0);
vector<double> xsec;
m_fnlo->CalcCrossSection();
xsec = m_fnlo->GetCrossSection();
```

7 Data Reading

A class Dataset is define for reading the data tables. And another class DataPoint is defined for saving all the information of one data point.

All the calculation is done through the Dataset class. Here is an example of working flow:

```
vector<Dataset *> AllData(1);

AllData.at(0) = new Dataset("ccfrf3.md");
AllData.at(0)->SetCut(setting);
AllData.at(0)->InputNormalization(1.0290);
AllData.at(0)->ReadFile("/Data/yaofu/FittingCode/da/data2021.0324/ccfrf3.md",
"DIS", "CTEQ", "", "ccfrf3.md");
AllData.at(0)->InitialPrediction(setting);
AllData.at(0)->GetPrediction();
AllData.at(0)->PrepareChi2();
AllData.at(0)->Chi2(true);
```

In the very beginning, the Dataset class reads the information from the data table, as well as the calculation mode. The DIS/VBP/AplGrid/FastNLO processes are all calculated through this class. The InitialPrediction() function initializes the PDF setting. Then the GetPrediction() function is called for doing the calculation. Finally, PrepareChi2() and Chi2() will be called for getting the χ^2 .

The DataPoint class saves all of the information of one data point, including errors, k-factors..... For DIS and VBP processes, a vector of DataPoint will be as an input giving to the CalcDIS and CalcVBP class.

```
m_calculation->CalcDIS::InitializeData(setting, m_data);
m_calculation->CalcVBP::InitializeData(setting, m_data);
```

The calculation in DIS and VBP class is based on the information saved in the DataPoint class:

```

for(int i = 0; i < result.size(); i++){

    if(m_datatable.at(i).LdatAdj >= 1) DatAdj(m_datatable.at(i));
    if(m_datatable.at(i).LHtg >= 1) HtgAdj(m_datatable.at(i));

    m_disdata = m_datatable.at(i);

    m_disdata.jBn = m_disdata.Jbsn;
    m_disdata.jHd = m_disdata.Jtgt;
    m_disdata.iLptn1 = m_disdata.JBem;

```

A global Config class is defined for saving global parameters, such as the cut and also the PDF information, see include/Calculation/Config.h. When doing the initialization for Dataset class, the Config class should be as an input:

```

virtual void InitialPrediction(Config setting);
virtual void InitialPDF(Config setting);

```

C. Multiple-thread

The DIS and VBP processes are using the multiple-thread technique to speed up the calculation. We use the POSIX thread to perform the multiple-thread. The main functions are pthread_create and pthread_join.

The pthread_create is the function for submitting jobs, and pthread_join is the function for waiting for the thread jobs finish.

Since some of data have amount of data points, and some of data only have several data points. To make the amount of calculation more average, in the very beginning we need to do reorganization for the existing data sets. This procedure makes each thread processes the same number of data points. This reorganization is done for DIS and VBP processes separately, since the speed of DIS and VBP calculation is also different. The speed of VBP processes is faster than the DIS processes, so it only needs less number of threads.

Here is the reorganization code:

```

// Reorganize DIS data
for(int idata = 0; idata < AllData.size(); idata++){
    if(AllData.at(idata)->DataType == "DIS"){
        for(int ipoint = 0; ipoint < AllData.at(idata)->NPoints; ipoint++){
            DISDataVector.push_back(AllData.at(idata)->GetDataPoint(ipoint));
        }
    }
}

nDISPointsPerThread = DISDataVector.size() / nDISThread;

int iThread = 0;
for(int ipoint = 0; ipoint < DISDataVector.size(); ipoint++){
    if(ipoint % nDISPointsPerThread == 0 && iThread != nDISThread) iThread++;
    ThreadDataDIS[iThread].push_back(DISDataVector.at(ipoint));
}

```

After the reorganization, we need to do the initialization for the reorganized Dataset:

```

// Initialize DIS Thread Jobs
for(int i = 1; i <= nDISThread; i++){
    ThreadRunDIS[i] = new Dataset();
    ThreadRunDIS[i]->SetDataTable(ThreadDataDIS[i]);
    ThreadRunDIS[i]->InitialPrediction(setting);
}

```

Then submit the multiple-thread jobs:

```

// Submit DIS Thread Jobs
for(int i = 1; i <= nDISThread; i++){
    pthread_create(&threadDIS[i], NULL, ThreadChi2, ThreadRunDIS[i]);
}

```

In the main thread, waiting for all the thread jobs finish:

```
// Waiting for DIS Jobs finish
for(int i = 1; i <= nDISThread; i++){
    pthread_join(threadDIS[i], NULL);
}
```

Here is the input function which gives pthread_create:

```
pthread_mutex_t ApplMutex = PTHREAD_MUTEX_INITIALIZER;

void* ThreadChi2(void* arg)
{
    Dataset* ptr = (Dataset *)arg;

    if(ptr->CalcMode == "ApplGrid"){
        if(pthread_mutex_lock(&ApplMutex) != 0){
            cout<<"Error: lock failed"<<endl;
        }
    }

    ptr->GetPrediction();

    if(ptr->CalcMode == "ApplGrid"){
        if(pthread_mutex_unlock(&ApplMutex) != 0){
            cout<<"Error: unlock failed"<<endl;
        }
    }

    pthread_exit(NULL);
}
```

Command between pthread_mutex_lock and pthread_mutex_unlock will run with single thread.

The multiple-thread for ApplGrid is through some other functions.

χ^2 calculation for sum of all data sets is done by an interface class called PDFChi2. To perform initialization and calculation of χ^2 , usage these two functions:

```
virtual void Initialize(vector<Dataset *> AllData, Config setting);
virtual void InitialPDF(Config setting);
```

D. Fitting

We have different kinds of minimizer implemented in the fitting code, which includes the ROOT TMinuit, Minuit2, BAT (optional and not recommended), and the TensorFlow.

The fitting consists mainly of three steps. An initialization function to be called for reading in all data files, ApplGrid files, and FastNLO files. And also the needed parameters and thread strategy are setup in the initialization function. A wrapper function which reads a set of parameters and output a χ^2 value, which is the main function to be called. A finalization function which reads a set of best fit parameters and errors to output all the updated theory prediction and PDFs. All those three functions are wrapped as Initialize(), chi2(), and Finalize() in src/Calculation/GlobalFittingCore.C. Those functions can be called either by TMinuit, Minuit2, and BAT through the C++ code, or by TensorFlow through the Python script.

See src/Calculation/GlobalFittingCore.C for details.

VI. Running the code

A. Running the best fit

We have different kinds of minimizer to perform the PDF global fit.

PDFFit

PDFFit_Minuit2

PDFFit_Minimizer

PDFFit_BAT

fitting.py

B. Running the error set

For running the iterate, usage

PDFEVSet

For reading the .ev file and generating the PDF error set, usage

ReadEV xxx.ev

C. Running the Lagrange Multiplier Scan

For running the PDF Lagrange Multiplier Scan, usage

PDFLMScan

For running the α_S LM scan, usage,

AlphaSLMScan
